Dragoon — Full Build Plan (Phase 0–10)

Local voice assistant · qwen3.5:4b + qwen3-embedding:0.6b · Python / Ollama / SQLite / Whisper / pyttsx3 · CPU-only, AMD Ryzen · Schedule basis: ~1–1.5 hrs/day, spread daily, with clustered sessions on stateful phases

Confidence key

[high] = verified against current sources/docs **[inference]** = reasoned estimate, not measured **[assumption]** = stated explicitly because the underlying data isn't known

0. Reality Check

1. "High precision probability of outcomes" is not something a CPU-only 4B model does. [high] What Phase 2 actually builds is **ranked candidates from scored heuristics** (embedding similarity + keyword overlap) — not calibrated statistics. Treat it as that internally, or you'll misjudge when it's working.
2. Every hour estimate below is a planning number, not a delivery date. [inference] Phase 2 and Phase 4 carry the most variance — scoring tuning and agent retry logic are where solo builds like this run long.
3. RAM: [assumption — unknown spec] confirm at least 8GB free before Phase 0. qwen3.5:4b is 3.4GB on disk; 8GB free is workable, 16GB is comfortable.
4. Schedule basis for every timeline figure below: ~1-1.5 hrs/day, spread daily (7 hrs/wk to 10.5 hrs/wk). [assumption — 'evenly' interpreted as every day] At this pace the honest range is **4.5-10.3 months**, with a realistic midpoint closer to **~7 months** than 6 — six months only happens at the low end of the hour estimate combined with the top of the pace range. Missing a day here and there (which will happen) pushes further toward the high end, not the low end.
5. Mitigation for the daily-fragment risk: Phase 2 (candidate scoring) and Phase 4 (agent loop) carry real state across a session — thresholds you're tuning, a state machine you're debugging. 1-hour fragments on these two specifically cost extra time re-loading context each session. Recommendation: run Phases 2 and 4 as clustered sessions (2-3x/week, 3+ hrs each) even while everything else stays on the daily 1-1.5 hr cadence. See the cadence column in Section 3.
6. Session-start ritual for every session, every phase: spend the first 5 minutes re-reading your last log entry / commit message before writing new code. Cheap insurance against the re-familiarization cost of a stretched timeline.

1. Model & Tooling

Model choice

Role	Model	Size	Notes
Primary LLM	qwen3.5:4b	3.4GB	256K context, native tool-calling, thinking mode available — disable thinking m
Embedding/scoring	qwen3-embedding:0.6b	~1GB	Used only in Phase 2 candidate scoring
Fallback (too slow)	qwen3.5:2b	2.7GB	Drop to this if Phase 0 latency test fails
Alt. if reasoning-limited	phi-4-mini-instruct	~2.5GB	Stronger reasoning/coding benchmarks; no confirmed data on tool-calling relia

Environment setup — execution commands

```
ollama pull qwen3.5:4b
ollama pull qwen3-embedding:0.6b
```

```
pip install ollama sounddevice numpy openai-whisper pyttsx3 --break-system-packages
# Windows only, if pyttsx3 SAPI5 import fails:
pip install pywin32 --break-system-packages
```

Repo structure

```
dragoon/
  core/
    intent.py         # Phase 1
    candidates.py      # Phase 2
    memory.py          # Phase 3
    agent_loop.py       # Phase 4
    hardening.py       # Phase 6
  tools/
    registry.py         # Phase 4/7
  io/
    stt.py
    tts.py
  data/
    dragoon.db
  logs/
  tests/
  README.md            # Phase 10
  main.py
```

Architecture

```
[Mic] -> [Whisper STT] -> [Intent Parser: qwen3.5:4b, JSON mode]
                                  |
                ----------------------------------
                |                                 |
        intent=command              intent=question/conversation
                |                                 |
[Multi-Candidate Gen: N=3, qwen3.5:4b]   [Direct response, memory-informed]
                |
[Score: qwen3-embedding:0.6b similarity + keyword overlap]
                |
      score >= threshold? --No--> [Ask user to disambiguate]
                |Yes
[Agent Loop: PARSE -> SELECT_TOOL -> EXECUTE -> VERIFY -> RESPOND]
                |
        [SQLite: read/write context]
                |
        [pyttsx3/SAPI5 TTS] -> [Speaker]
```

Definition of Done (applies to every phase)

A phase is complete when its exit criteria are met **and reproduced on a second, independent test run** — not on the first pass. Sampling-based models look done more often than they are; re-verification is part of the definition, not an extra step.

2. Known Risks, Mapped to the Phase That Addresses Them

Risk	Addressed in
RAM pressure / swapping under combined LLM+embedding+Whisper load	Phase 0 (measure), Phase 8 (fix)
Thermal throttling on sustained CPU load	Phase 0, Phase 9 (soak test surfaces this)

Quantization artifacts: malformed JSON, misclassification	Phase 1, Phase 6
Overconfident wrong intent classification (silent, not flagged)	Phase 1 exit criteria — test for confident-wrong, not just wrong
Low-diversity candidates at moderate temperature	Phase 2
Embedding brittleness on novel phrasing	Phase 2
Thinking mode activating on latency-sensitive calls	Phase 8
Validation-execution race in tool calls	Phase 4 — validate before EXECUTE, never after
State bleed between commands in the agent loop	Phase 4
VERIFY checking the wrong condition (loop 'succeeds' incorrectly)	Phase 4, Phase 6
Bad STT transcription cascading into everything downstream	Phase 9 (real background noise testing)
pyttsx3/SAPI5 driver stopping output after repeated calls	Phase 9 — fix: reinitialize engine fresh per call
Compounding latency across sequential pipeline stages	Phase 8 — the single highest-leverage phase on this list
Memory retrieval heuristic dropping relevant / keeping stale context	Phase 3, Phase 8
Ollama tag drift (model behind a tag changes silently later)	Phase 10 — pin exact digest, not just the tag

Phase 0 — Environment & Model Validation

Objective: Confirm the model runs acceptably on your hardware before building anything on top of it.

1. Confirm Ollama is current: `ollama --version`, update if needed.
2. Pull both models (Section 1 commands).
3. Write a test script sending 10 varied prompts: plain Q&A, a JSON-extraction prompt, a tool-call-style prompt.
4. Log latency with time.time() deltas — measure first-token time and total time separately.
5. Record tokens/sec and RAM usage during inference (psutil or Task Manager).
6. Confirm tool-call/JSON output parses as valid JSON on ≥8/10 prompts.
7. Decision gate: if average response time exceeds ~15-20s for a short prompt, drop to qwen3.5:2b and re-run steps 3-6.

Exit criteria: Documented latency numbers + ≥8/10 prompts producing parseable structured output, reproduced on a second run.

Estimated hours: 8-12 hrs [inference] | Cadence: Daily light OK (1-1.5 hrs/day)

Phase 1 — Intent Parser

Objective: Single-call classifier: command / question / conversation, structured output only.

1. Define output schema: {"intent": "command|question|conversation", "raw_text": "string"}.
2. Use Ollama's format:"json" parameter — don't rely on prompt wording alone to enforce JSON.
3. Write 60-100 labeled test utterances yourself (20-30 per category), including deliberately ambiguous ones.
4. Run the classifier against the set, compute accuracy.
5. Iterate prompt/few-shot examples until accuracy is stable across two separate runs.

Exit criteria: ≥85% accuracy, reproduced on a second independent run.

Estimated hours: 15-25 hrs [inference] | Cadence: Daily light OK (1-1.5 hrs/day)

Phase 2 — Multi-Candidate Generation + Scoring

Objective: For command-type input, generate multiple interpretations and rank them. This is the 'probability' layer — it is a scored ranking, not calibrated probability.

1. Sample N=3 completions at temperature ~0.7-0.9 for classified commands.
2. Embed each candidate + known command vocabulary with qwen3-embedding:0.6b; compute cosine similarity (numpy).
3. Score = 0.6 × embedding similarity to nearest known command + 0.4 × keyword overlap.
4. Threshold (start at 0.75, tune empirically): above -> auto-execute top candidate; below -> voice-prompt disambiguation.
5. Log every low-confidence case to a file — this becomes your tuning dataset.

Exit criteria: On a 40-command mixed test set: ≥90% correct auto-execution on clear commands, zero silent wrong-executions on ambiguous ones.

Estimated hours: 35-50 hrs [inference] | Cadence: Cluster: 3+ hr sessions, 2-3x/week

Phase 3 — Structured Memory

Objective: Replace flat chat history with queryable state.

1. Create SQLite schema (commands, context, preferences tables — see code block below).
2. Write get_context(n=5) — last N commands + context rows updated in the last 24h.
3. Write update_context(key, value) — upsert with timestamp.
4. Inject retrieved context as a bounded block (cap ~500 tokens) into the prompt.
5. Test staleness: manually insert an old context row, confirm the retrieval window correctly excludes/deprioritizes it.

```
CREATE TABLE commands (id INTEGER PRIMARY KEY, ts TEXT, raw_text TEXT, intent TEXT, outcome TEXT);
CREATE TABLE context (key TEXT PRIMARY KEY, value TEXT, updated_at TEXT);
CREATE TABLE preferences (key TEXT PRIMARY KEY, value TEXT);
```

Exit criteria: Memory-informed responses correctly reference a fact stated 5+ turns earlier, in ≥8/10 manual test conversations.

Estimated hours: 20-30 hrs [inference] | Cadence: Daily light OK (1-1.5 hrs/day)

Phase 4 — Agent / Tool Loop (core)

Objective: A state machine that takes actions, not one that talks about taking them.

1. Define states explicitly as an enum: PARSE -> SELECT_TOOL -> EXECUTE -> VERIFY -> RESPOND.
2. Tool registry as a plain dict: {"tool_name": callable}. Start with 3-4 low-risk tools (reminder to SQLite, read system time, open a local file, simple calculation). No destructive tools yet.
3. SELECT_TOOL: model outputs tool name + args as JSON, validated against a manual per-tool schema.
4. EXECUTE: call the function, catch exceptions explicitly.
5. VERIFY: a cheap deterministic check (did the expected file/DB row appear) — not a second LLM call.
6. On VERIFY failure: retry once with the error fed back to the model; on second failure, stop and ask the user directly.

Exit criteria: 20 consecutive tool-invoking commands, zero silent failures, zero loops past the retry cap.

Estimated hours: 45-65 hrs [inference] | Cadence: Cluster: 3+ hr sessions, 2-3x/week

Phase 5 — Integration + Stress Test

Objective: Full pipeline, real conditions, first end-to-end pass.

1. Wire STT -> intent parser -> candidate scoring -> memory-informed prompt -> agent loop -> TTS end to end.
2. Run a 20-30 turn continuous session: clear commands, ambiguous commands, questions, casual conversation, 3+ adversarial/nonsense inputs.
3. Log per-stage latency separately, not just total.
4. Fix state-machine edge cases and re-tune thresholds based on what breaks.

Exit criteria: Full session completes with no crash, no silent wrong-execution, and per-stage latency logs you can act on.

Estimated hours: 20-30 hrs [inference] | Cadence: Daily light OK (1-1.5 hrs/day)

Phase 6 — Hardening & Error Recovery

Objective: Survive bad input, device failures, and edge cases without crashing or behaving unpredictably.

1. Enumerate failure modes per component: mic not found, empty/garbage transcription, Ollama server unresponsive, malformed JSON despite format=json, SQLite lock/write failure, TTS engine failure.
2. Wrap every external call (Ollama, SQLite, audio device, TTS) in explicit try/except with typed handling — no bare except.
3. Add a global fallback response for any unhandled state, so the assistant never goes silently unresponsive.
4. Build structured logging (Python logging module, rotating file) — every error, retry, and low-confidence routing decision.
5. Deliberately inject failures in testing: kill the Ollama server mid-call, disconnect the mic, corrupt a DB write.

Exit criteria: 15 deliberately-injected failure scenarios, zero unhandled crashes, all logged with enough detail to diagnose after the fact.

Estimated hours: 15-25 hrs [inference] | Cadence: Daily light OK (1-1.5 hrs/day)

Phase 7 — Tool Expansion & Safety Sandboxing

Objective: Move beyond the starter tools safely, including any higher-risk actions.

1. Define a risk tier per tool: low (read-only), medium (file writes in a sandboxed folder), high (system commands, deletions).
2. For medium/high tools, add an explicit confirmation step: state the action out loud, require clear affirmative before EXECUTE.
3. Sandbox file-writing tools to one dedicated directory — never allow arbitrary path writes from model output directly.
4. Add per-tool argument whitelisting (e.g., a calculation tool rejects anything not a parseable expression — no eval() on raw strings).
5. Expand tool registry to 8-10 tools; re-run the Phase 4 exit criteria against the larger registry.

Exit criteria: Zero unconfirmed medium/high-risk executions across 30 test commands; no tool accepts unsanitized input.

Estimated hours: 15-25 hrs [inference] | Cadence: Daily light OK (1-1.5 hrs/day)

Phase 8 — Performance Optimization & Latency Tuning

Objective: Address compounding latency directly — sequential candidate generation + embedding + memory retrieval + agent loop adds up fast.

1. Profile end-to-end latency per stage across the full pipeline, on 20 real test commands.
2. Identify the single largest contributor (commonly Phase 2 candidate generation) and address it first: reduce N from 3 to 2 for lower-ambiguity intents, or skip multi-candidate entirely for high-confidence single-interpretation commands.
3. Cache repeated embedding lookups for known vocabulary (compute once at startup).
4. Trim injected memory context to the minimum that still passes Phase 3's exit criteria.
5. Explicitly disable thinking mode on every latency-sensitive call — confirm via logged output it's actually off.
6. Re-measure; set a defined max-latency ceiling and treat exceeding it as a bug, not a known limitation.

Exit criteria: Documented before/after latency showing measurable improvement, and a stated latency ceiling met on ≥90% of test commands.

Estimated hours: 12-20 hrs [inference] | Cadence: Daily light OK (1-1.5 hrs/day)

Phase 9 — Extended Real-World Testing (soak test)

Objective: Validate under real conditions over real elapsed time — this phase has a calendar floor independent of hours/day.

1. Run across multiple real sessions over at least 5-7 calendar days — cannot be compressed into one long day.
2. Test with real background noise (not your quiet test setup) for STT accuracy.
3. Let at least two sessions run long (30+ minutes), specifically to catch the pyttsx3/SAPI5 long-session driver issue and any memory/context creep.
4. Track every failure in the Phase 6 structured log; categorize by root cause (STT, classification, scoring, tool execution, TTS).
5. Fix anything recurring more than twice; log-and-defer true one-offs.

Exit criteria: 5+ calendar days of real use, no unresolved recurring failure category, TTS confirmed stable across two 30+ minute sessions.

Estimated hours: 12-20 hrs [inference] | Cadence: Daily light OK (1-1.5 hrs/day)

Phase 10 — Documentation & Packaging

Objective: Make the project usable by you in six months, not just today.

1. Write a README: setup steps, exact model versions/tags, hardware requirements, known limitations stated plainly.
2. Document the config format (thresholds, tool registry, memory retention window) so tuning doesn't require reading source.
3. Pin the exact Ollama model digest used, not just the :4b tag, so a future pull can't silently change behavior under you.
4. Package as a single entry point (main.py with clear CLI args) instead of manual script edits to run.
5. Tag a version at this point — this is v1.0, the actual deliverable, not another phase.

Exit criteria: A stranger — or you, in six months — could set this up from the README alone without asking you anything.

Estimated hours: 6-10 hrs [inference] | Cadence: Daily light OK (1-1.5 hrs/day)

3. Timeline — Phase 0 to Phase 10

Basis: ~1-1.5 hrs/day, spread every day (7-10.5 hrs/wk). Cadence column shows where daily fragments work fine vs. where clustering into fewer longer sessions is recommended instead — see the Mitigation note in Section 0. This is NOT a 6-month-guaranteed schedule; see totals below.

Phase	Hours	Weeks @ 1 hr/day (7 hrs/wk)	Weeks @ 1.5 hrs/day (10.5 hrs/wk)	Cadence
0 — Environment & Model Validation	8-12	1.1-1.7	0.8-1.1	Daily light OK
1 — Intent Parser	15-25	2.1-3.6	1.4-2.4	Daily light OK
2 — Multi-Candidate Generation + Scoring	35-50	5.0-7.1	3.3-4.8	Cluster: 3+ hr sessions, 2-3x/week
3 — Structured Memory	20-30	2.9-4.3	1.9-2.9	Daily light OK
4 — Agent / Tool Loop (core)	45-65	6.4-9.3	4.3-6.2	Cluster: 3+ hr sessions, 2-3x/week
5 — Integration + Stress Test	20-30	2.9-4.3	1.9-2.9	Daily light OK
6 — Hardening & Error Recovery	15-25	2.1-3.6	1.4-2.4	Daily light OK
7 — Tool Expansion & Safety Sandboxing	15-25	2.1-3.6	1.4-2.4	Daily light OK
8 — Performance Optimization & Latency Tuning	12-20	1.7-2.9	1.1-1.9	Daily light OK
9 — Extended Real-World Testing (stress test)	12-20	1.7-2.9	1.1-1.9	Daily light OK
10 — Documentation & Packaging	6-10	0.9-1.4	0.6-1.0	Daily light OK
TOTAL	**203-312**	**29.0-44.6 wks (~6.7-10.3 mo)**	**19.3-29.7 wks (~4.5-6.9 mo)**	—

Note on Phase 9: its hours are low relative to other phases, but it has a hard calendar floor of 5-7 elapsed days regardless of hours/day — factor that in separately, it doesn't compress.

If six months is a hard ceiling, not a target: the realistic midpoint above (~7 months) means something has to give. The lowest-cost cut is Phase 7 (Tool Expansion) — the core loop is already fully functional and demoable after Phase 5/6 with 3-4 tools; Phase 7's extra 15-25 hrs adds breadth, not core capability. Cutting it first, before touching Phase 8's latency work or Phase 9's real-world testing, preserves the parts that actually make this portfolio-worthy.

First Checkpoint (this week)

Finish Phase 0, all 7 steps, both exit criteria met on two separate runs, before writing any pipeline code. If step 6 (JSON/tool-call parsing) fails on more than 2/10 prompts, that's the real starting problem — fix parsing before touching Phase 1.